

TeddyTales — Т3: Анимация персонажа-медведя (Rive + Flutter)

Статус: черновик для передачи в Claude Code **Автор:** Руслан (техдиректор проекта TeddyTales) **Дата:** август 2026

0. Границы документа

Это Т3 покрывает **только** пайплайн создания и интеграции анимированного персонажа-медведя: риг в Rive, State Machine, Data Binding, MCP-воркфлоу и интеграцию в Flutter.

Не входит в этот документ (описано в других материалах проекта TeddyTales):

- Общая бизнес-логика приложения (магазин, мини-игры, обучение, e-commerce)
 - Backend/бэкенд-стек — **не подтверждён** (Flutter + Supabase рассматривался, но решение отложено). Claude Code не должен предполагать конкретный backend, пока это не подтверждено отдельно.
 - Точный набор игровых характеристик мишки (голод/настроение/декей) — **открытый вопрос**, см. раздел 8.
-

1. Контекст

TeddyTales — Tamagotchi-style мобильное приложение (виртуальный медведь): кастомизация комнаты, магазин, стадии роста, дневник, заказ физической плюшевой игрушки на финальной стадии. Полная спецификация приложения — в отдельных документах проекта.

Эта часть Т3 касается конкретно **анимированного персонажа** — самого медведя на экране, его визуального поведения и реакций.

2. Выбранный подход к анимации

- **2D скелетная анимация в Rive** — не 3D, не Unity, не motion capture, не покадровое видео.
 - Причина выбора: лёгкие файлы, нативная интеграция с Flutter через официальный runtime, интерактивность в реальном времени (state machine реагирует на данные приложения), низкий порог входа для не-художника.
 - **Риг и State Machine собирает Руслан лично** в Rive Editor.
 - **Художник поставляет только отдельные слои** (тело, голова, уши, лапы, глаза и т.д.) — не готовую анимацию.
 - Тариф Rive: **Cadet, \$9/мес** — даёт безлимитный экспорт `.riv` для продакшена и Early Access (нужен для MCP).
-

3. Структура рига — референс именования

Референс: `github.com/2d-inc/Flare-Flutter`, папка `example/teddy`. Старый формат Flare (`.flr`), сам файл не переиспользуется напрямую — но структура именования костей и узлов проверена в проде и должна лечь в основу нашего рига.

3.1 Кости (Bones)

```
root
root_body → body, body_base, head, head_shadow
root_arm_left / root_arm_right → forearm → forearm_light → hand → hand_light
    → finger_1_nail, finger_2_nail, finger_3_nail
```

3.2 Уши (пример детализации на объект)

```
ear_left / ear_right
ear_in_left / ear_in_right      (внутренняя часть уха, отдельный слой)
ear_light_left / ear_light_right (блик, отдельный слой)
```

3.3 Лицо

```
eye_left / eye_right
pupil_left / pupil_right
pupil_light_left / pupil_light_right
eyebrow_left / eyebrow_right
eyelid_top_left / eyelid_top_right
```

eyelid_bottom_left / eyelid_bottom_right
nose, mouth, teeth, tongue, lips

3.4 Control-узлы (невидимые “рычаги” для управления группами частей)

ctrl_face — общий контроль лица
ctrl_eyes — направление взгляда (двигает оба глаза разом)
ctrl_pupils — контроль зрачков
ctrl_mouth — управление ртом
ctrl_nose — управление носом
ctrl_eyebrow_left / ctrl_eyebrow_right

3.5 Аксессуары (пример из референса — сегментированный шарф)

scarf_1 / scarf_2 / scarf_3 — посегментно, для физики покачивания

Требование к художнику: каждая часть — отдельный слой с именем по этой конвенции (или максимально близким аналогом). Конвенцию именования зафиксировать в Rive: **Workspace → Admin → Options → Naming convention → snake_case**.

4. Rive MCP + Claude Code — workflow

4.1 Что такое Rive MCP

Официальный MCP-сервер, встроенный в Rive Editor **Early Access** (не в Production-версию редактора). Позволяет управлять открытым файлом Rive через естественный язык из внешнего AI-инструмента.

4.2 Подключение

Сервер — локальный HTTP-эндпоинт: `http://127.0.0.1:9791/mcp`, поднимается только пока открыт Rive Editor (Early Access) **на том же компьютере**, где работает Claude Code.

Способ 1 — GUI (Claude Code desktop app), рекомендуемый для работы через чат:

Настройки → Connectors → “+” → Add custom connector → вставить Name + URL
(`http://127.0.0.1:9791/mcp`) → Add.

Способ 2 — терминал (запасной, если GUI не примет локальный адрес):

```
claude mcp add --transport http rive http://127.0.0.1:9791/mcp
```

4.3 Что MCP реально умеет (подтверждено официально)

- Создание и редактирование **View Models** (свойства данных)
- Создание **State Machine** (состояния, переходы, слои)
- Работа с **Layouts** и **Shapes**
- Настройка градиентов на заливках/обводках
- Component data context (привязка Model/Path компонентов)
- Relative data binding через вложенные view model'и

4.4 Что MCP НЕ подтверждено как умеющий (и что остаётся ручной работой)

- **Расстановка костей/скелета (bone rigging) на импортированных слоях** — нет задокументированных примеров, что MCP это делает надёжно. Аналогичная ситуация в куда более зрелом Blender MCP: официально подтверждено, что риггинг с покраской весов и IK "нереалистичен с текущими возможностями MCP".
- Финальная "полировка" ощущения живости анимации (timing, вес движения) — визуальная ручная работа.

Вывод для порядка работ: риг строится вручную в редакторе → **затем** MCP/скриптинг подключаются для View Model, State Machine и поведенческой логики поверх готового рига.

5. State Machine / View Model — шаблон (требуется финального подтверждения)

Точный набор характеристик мишки — **открытый вопрос проекта** (decay-таймеры, механика пренебрежения/болезни ещё не подтверждены Русланом). Ниже — рабочий шаблон по аналогии с официальным примером Rive (Sasquatch), где переменная `anger` росла от тапов и затухала со временем, управляя blend state между анимациями рта.

По той же механике, но для мишки:

View Model properties (пример, не финал):

```
hunger      (number, 0-100)
mood         (number, 0-100)
growthStage  (number или enum)
```

State Machine states (пример):

```
idle
happy
sad
eating
tap_reaction
growth_transition
```

Blend state:

```
mouth/face expression = blend(sad_pose, happy_pose, weight: mood)
```

Паттерн скриптинга (Lua + AI-агент Rive) — по прямой аналогии с официальным туториалом Sasquatch (rive.app/blog/scripting-with-the-ai-coding-agent):

1. Скрипт с input-переменной, привязанной к View Model property
2. Decay-логика: значение падает со временем (по секундам)
3. Триггер по тапу/действию: значение растёт
4. Blend state или transition, управляемый этим значением

Референс-файлы (открыть, разобрать структуру, повторить паттерн на мишке):

- Стартовый файл: rive.app/community/files/25775-48125-sasquatch-starter-file
- Финальная версия: rive.app/community/files/25774-48147-interactive-sasquatch

6. Flutter-интеграция

6.1 Пакет

Использовать **официальный** `rive-flutter` (github.com/rive-app/rive-flutter), НЕ `rive-flutter-legacy`.

6.2 Базовое подключение

```
import 'package:rive/rive.dart';
```

```
RiveAnimation.asset(  
  'assets/bear.riv',  
  stateMachines: 'State Machine 1',  
)
```

6.3 Управление из кода приложения

Через `stateMachineInputs` — установка значений (`hunger`, `mood`) или вызов триггеров (`bumpTrigger.fire()`), приходящих из игровой логики приложения.

6.4 Паттерн контроллера

За основу структуры взять `teddy_controller.dart` из референса (`2d-inc/Flare-Flutter/example/teddy/lib/teddy_controller.dart`):

- хранение ссылок на control-узлы
- метод `play(String animationName)`
- реакция на внешние события (например, `coverEyes(bool)`, `submitPassword()` в референсе → по аналогии `feedBear()`, `petBear()` и т.д. для нашего кейса)

6.5 Известная техническая проблема

Начиная с Flutter 3.10 рендерер **Impeller** заменил Skia по умолчанию на iOS — возможны визуальные расхождения между Rive Editor и приложением. При багах: запустить с флагом `--no-enable-impeller`, сравнить со Skia-рендером, прежде чем заводить баг-репорт.

7. Дополнительные референсы

- **Официальный репозиторий Rive:** github.com/rive-app, ключевые репо — `rive-flutter` (runtime), `rive-runtime` (низкоуровневый C++ движок), `rive-docs`
- **Пример state machine с idle-анимацией персонажа (Marketplace):**
rive.app/marketplace/16143-30395-character-animate-state-machine/
- **Референс игровой логики Tamagotchi-механики** (другой стек — Tauri+React, НЕ Rive/Flutter, но полезная архитектура для decay/growth-stage логики и структуры MCP-доступа к состоянию питомца): github.com/siegerts/tama96
- **Кейс от Rive:** ментальное здоровье-приложение с абстрактным "buddy"-персонажем, реагирующим на ввод пользователя в реальном времени, включая

eye tracking — аналогичный паттерн поведения (medium.com , "Bringing our mental health app to life with Rive")

8. Открытые вопросы (не предполагать, уточнить у Руслана)

1. Точный список характеристик мишки и их decay-таймеры (голод? настроение? чистота? энергия?)
 2. Механика пренебрежения / "болезни" питомца — есть ли она вообще
 3. Итоговый backend-стек (Flutter + Supabase не подтверждён)
 4. Полный список стадий роста и триггеров перехода между ними
 5. Нужна ли физика (покачивание ушей/шарфа) на MVP-этапе или это уже пост-MVP полировка
-

9. Первая итерация задач для Claude Code

Когда репозиторий будет создан и Claude Code подключён:

1. Создать структуру Flutter-проекта / модуля под виджет мишки (если ещё не создан)
 2. Написать класс-контроллер по образцу `teddy_controller.dart` , с заглушками под `hunger / mood` (значения-плейсхолдеры до финального ответа по п.8.1)
 3. Подключить пакет `rive` (не legacy), настроить `RiveAnimation.asset` с загрузкой `.riv` -файла (файл появится после того, как риг будет собран в редакторе)
 4. Подготовить точку интеграции `stateMachineInputs` — методы для передачи значений из игровой логики приложения в риг
 5. Не приступать к остальной бизнес-логике (магазин, e-commerce, мини-игры) — вне границ этого ТЗ
-

Документ будет обновлён после ответов на открытые вопросы (раздел 8) и по факту готовности первых слоёв от художника.

